

ContextGraph

Problem Statement

1. Introduction

Developers and technical professionals increasingly work across several AI assistants and AI-assisted tools — Claude, ChatGPT, Cursor, Codex — often within the same day, and across several ongoing projects or clients. Each of these tools maintains its own notion of conversational memory, if it maintains one at all, and none of them share that memory with one another.

2. Problem Statement

Every new AI session starts from zero context. A user must repeatedly re-explain who they are, what technology stack a given project uses, what has already been decided or ruled out, and what constraints apply — architectural choices, client-specific rules, coding conventions. This repetition costs time, breaks flow, and produces inconsistent answers across tools because each restatement is slightly different from the last.

The AI vendors that do offer a memory feature (ChatGPT Memory, Claude's Projects/memory) address only part of this problem, and each has structural limitations:

- **Siloed** — memory captured inside one AI product is invisible to any other AI product.
- **Non-portable** — there is no standard way to export a user's accumulated context and carry it to a different tool.
- **Flat, not hierarchical** — existing implementations store one undifferentiated pool of facts. They cannot express that a preference is global, a constraint is specific to one client, and a rule is specific to one project. Without this containment, instructions from one context leak into an unrelated one.
- **Silent, not user-directed** — memory is inferred by the vendor's model with no explicit save action and no visibility into why something was kept or dropped.

3. Objectives

Build a graph-structured, user-owned context store that:

- Is reachable by any MCP-compatible AI client through a single protocol and a personal API key.
- Represents context hierarchically — global, agency/client, and per-project scopes.
- Lets an explicit user action (/save) trigger an AI judgment step that decides what is actually worth remembering, with a visible relevance score that decays over time.
- Gives the user a visual dashboard to inspect, edit, and prune what has been stored.

4. Scope

In scope (V1)	Deferred (post-V1)
Email/password + Google auth	Memory export parsing as a first-class import
Structured onboarding wizard → initial graph	Multiple scoped API keys per user

In scope (V1)	Deferred (post-V1)
Dashboard with 3D graph visualization	Node version history
Node create / edit / delete	Public read-only graph sharing
MCP endpoint: get_context, save_context, list_nodes	Browser extension for web chat UIs
API key generation and per-client connection snippets	
Weekly relevance decay	
Dark and light mode	

5. Target Users

- **Primary** — developers and AI power users who use multiple AI tools daily and work across multiple concurrent projects.
- **Secondary** — freelancers and agency owners managing distinct context per client.

6. Success Criteria

The problem is considered solved when a user can:

- 1 Sign up and complete onboarding.
- 2 See their context graph in the dashboard.
- 3 Connect any MCP-compatible AI using their personal API key.
- 4 Start a session where get_context loads their context automatically.
- 5 Type /save and see the new entry appear in the dashboard graph.